Software Engineering (10th ed.) — Chapter 4: Requirements Engineering

Answers to Exercises 4.1–4.10

Nguyn Đình Thiên Lc

November 1, 2026

Section references (e.g. Sec. 4.1.2) point to the sections of Chapter 4 of I. Sommerville, Software Engineering*, 10th ed., where the supporting material is found.*

Contents

Exercise 4.1

Exercise 4.1

Identify and briefly describe four types of requirements that may be defined for a computer-based system.

Answer.

1. **User requirements** – statements in natural language plus diagrams of the services the system provides and the constraints under which it operates, written for customers and end-users. *[Ref. Sec. 4 (intro)]*

2. **System requirements** – more detailed descriptions of the system's functions, services and operational constraints; they define exactly what is to be implemented and may form part of the contract. *[Ref. Sec. 4 (intro)]*

3. **Functional requirements** – statements of the services the system should provide, how it should react to particular inputs and behave in particular situations (and sometimes what it should not do). *[Ref. Sec. 4.1.1]*

4. **Non-functional requirements** – constraints on the services or the system as a whole, such as timing, reliability, security, usability, and constraints imposed by standards. They are classified as *product*, *organizational* and *external* requirements. *[Ref. Sec. 4.1.2]*

Exercise 4.2

Exercise 4.2

Discover ambiguities or omissions in the following statement of the requirements for part of a drone system intended for search and recovery:

The drone, a quad chopper, will be very useful in search and recovery operations, especially in remote areas or in extreme weather conditions. It will click high-resolution images. It will fly according to a path preset by a ground operator, but will be able to avoid obstacles on its own, returning to its original path whenever possible. The drone will also be able to identify various objects and match them to the target it is looking for.

Answer. Natural-language requirements suffer from lack of clarity, confusion between functional and non-functional requirements, and amalgamation of several requirements in one sentence *[Ref. Sec. 4.4.1]*. Problems found:

1. *"very useful"* – a goal, not a verifiable requirement *[Ref. Sec. 4.1.2]*.

2. *"remote areas"* – maximum operating range from the ground station is not stated; nor is the communication link used when out of radio range.

3. *"extreme weather conditions"* – undefined: which wind speeds, temperatures, rain/snow levels?

4. *"high-resolution images"* – no resolution, frame rate, still vs. video, day/night (thermal) capability; how often are images taken and where are they stored or transmitted?

5. *"a path preset by a ground operator"* – how is it specified (waypoints, altitude)? Can it be changed during flight? Can the operator take manual control?

6. *"avoid obstacles"* – which obstacles (static, moving), at what minimum distance? *"whenever possible"* – what happens when it is not possible (hover, return home, alert operator)?

7. *"identify various objects"* – which object classes? How is the target described to the drone? Required accuracy / false-positive rate? What does the drone do after a match (notify, hover, mark GPS position)?

8. **Omissions**: flight endurance and battery behaviour (low-battery return), loss of communication behaviour, maximum altitude/speed, payload (e.g. dropping supplies for "recovery"), safety and legal constraints (aviation regulations) *[Ref. Sec. 4.1.2]*, and response times.

Exercise 4.3

Exercise 4.3

Rewrite the above description using the structured approach described in this chapter. Resolve the identified ambiguities in a sensible way.

Answer. Using the standard form of a structured specification *[Ref. Sec. 4.4.2]* (function, description, inputs, source, outputs, destination, action, requirements, pre/post-conditions, side effects), with ambiguities resolved by reasonable assumptions:

Drone/SR/01 – Follow preset flight path

Function	Follow flight path
Description	Fly the quad-chopper along a route defined by the ground operator.
Inputs	Ordered list of waypoints (latitude, longitude, altitude 30–120 m), cruise speed (≤ 15 m/s).
Source	Ground control station (GCS) operator.
Outputs	Current position, heading, battery level (telemetry at 1 Hz).
Destination	GCS.
Action	The drone flies from waypoint to waypoint. The operator may upload a new path or take manual control at any time. If the GCS link is lost for more than 30 s, or the battery falls below 25%, the drone returns to the launch point.
Requires	Valid path uploaded; GPS fix; battery $\geq 80\%$ at take-off.
Pre-condition	Pre-flight self-test passed.
Post-condition	Drone has landed at launch point or at an operator-specified landing point.
Side effects	None.

Drone/SR/02 – Avoid obstacles

Function	Obstacle avoidance
Description	Detect and avoid static or moving obstacles on the flight path.
Inputs	Proximity/vision sensor readings.
Source	On-board sensors (range ≥ 20 m).
Outputs	Avoidance manoeuvre; obstacle event message.
Destination	Flight controller; GCS log.
Action	When an obstacle is detected within 10 m on the path, the drone deviates to keep a clearance of at least 5 m, then rejoins the original path at the nearest point beyond the obstacle. If no safe route is found within 30 s, it hovers and requests operator instructions.
Requires	Sensors operational.
Pre-condition	Drone is in autonomous flight.
Post-condition	Drone is back on the original path, or hovering awaiting the operator.
Side effects	Mission time increases.

Drone/SR/03 – Capture images	
Function	Capture images
Description	Record imagery of the search area.
Inputs	Camera and thermal sensor data.
Source	On-board camera (minimum 12 MP stills, 4K/30 fps video) and thermal camera.
Outputs	Geo-tagged images, every 2 s or on operator command.
Destination	On-board storage; low-resolution preview streamed to GCS.
Action	Images are tagged with GPS position, altitude and time and stored; a preview is transmitted to the GCS.
Requires	At least 2 GB free storage.
Pre-condition	Drone airborne.
Post-condition	Images stored.
Side effects	None.

Drone/SR/04 – Identify target	
Function	Identify target objects
Description	Recognize objects in images and match them to the search target.
Inputs	Captured images; target description (class: person, vehicle, boat; colour; reference photo).
Source	Drone/SR/03; GCS operator.
Outputs	Match alert with confidence score, image and GPS location.
Destination	GCS.
Action	Each image is classified; any object of the target class with a match confidence $\geq 80\%$ raises an alert and the drone hovers over the location until the operator confirms or rejects the match.
Requires	Target description uploaded before or during the mission.
Pre-condition	Drone/SR/03 active.
Post-condition	Alert acknowledged by operator.
Side effects	Mission path is paused while waiting.

Environmental constraint (non-functional): the drone shall operate in winds up to 40 km/h, in rain up to 10 mm/h, and at temperatures from -20°C to $+45^{\circ}$C, at up to 5 km from the GCS.

Exercise 4.4

Exercise 4.4

Write a set of non-functional requirements for the drone system, setting out its expected safety and response time.

Answer. Non-functional requirements should be written quantitatively so they can be tested, using metrics such as speed, reliability or robustness *[Ref. Sec. 4.1.2]*.

Safety (product requirements)

NFR-S1 If the communication link with the GCS is lost for more than 30 s, the drone shall automatically return to its launch point.

NFR-S2 When battery charge falls below 25%, the drone shall abort the mission and return; below 10% it shall perform an immediate controlled landing.

NFR-S3 The drone shall maintain a minimum separation of 5 m from any detected obstacle and 30 m from people not designated as targets.

NFR-S4 The drone shall not exceed an altitude of 120 m above ground level nor enter geo-fenced no-fly zones (external/legislative requirement).

NFR-S5 A manual operator override command shall take precedence over all autonomous behaviour.

NFR-S6 The probability of an uncontrolled crash shall be less than 1 per 10,000 flight hours.

Response time (performance)

NFR-R1 The drone shall start an avoidance manoeuvre within 200 ms of detecting an obstacle.

NFR-R2 Operator commands shall be executed within 500 ms of being sent from the GCS.

NFR-R3 A target-match alert shall reach the GCS within 2 s of the image being captured.

NFR-R4 The live video preview latency at the GCS shall not exceed 1 s.

NFR-R5 Telemetry (position, battery) shall be updated at the GCS at least once per second.

NFR-R6 The drone shall be ready for take-off within 3 minutes of power-on.

Exercise 4.5

Exercise 4.5

Using the technique suggested here, where natural language descriptions are presented in a standard format, write plausible user requirements for the following functions:

- An unattended petrol (gas) pump system that includes a credit card reader. The customer swipes the card through the reader, then specifies the amount of fuel required. The fuel is delivered and the customer's account debited.
- The cash-dispensing function in a bank ATM.
- In an Internet banking system, a facility that allows customers to transfer funds from one account held with the bank to another account with the same bank.

Answer. Guidelines followed *[Ref. Sec. 4.4.1]*: a standard format, "shall" for mandatory and "should" for desirable requirements, one requirement per statement, plus a rationale.

(a) Unattended petrol pump

PUMP-1 The system **shall** allow the customer to pay by swiping a credit or debit card in the card reader.
Rationale: the pump is unattended, so no cashier is available.

PUMP-2 The system **shall** validate the card with the card issuer and reject expired, blocked or unreadable cards with an explanatory message.

PUMP-3 After card validation the system **shall** ask the customer to enter the amount of fuel, either as a volume (litres) or a monetary value, up to a pre-authorized maximum.

PUMP-4 The system **shall** pre-authorize the requested amount on the card account before enabling the pump.
Rationale: prevents delivery of fuel that cannot be paid for.

PUMP-5 The system **shall** stop delivery automatically when the requested amount is reached or the nozzle is replaced.

PUMP-6 The system **shall** debit the customer's account with the cost of the fuel actually delivered, not the requested amount.

PUMP-7 The system **should** offer a printed or e-mailed receipt.

(b) ATM cash dispensing

ATM-1 The ATM **shall** dispense cash only after the customer has inserted a valid card and entered the correct PIN.

ATM-2 The ATM **shall** allow the customer to select a predefined amount or enter an amount that is a multiple of the smallest note available.

ATM-3 The ATM **shall** refuse a withdrawal that exceeds the account balance/overdraft limit or the daily withdrawal limit, and inform the customer of the reason.

ATM-4 The ATM **shall** return the card before dispensing the cash.
Rationale: reduces the number of cards left in the machine.

ATM-5 The ATM **shall** debit the account only after the cash has been successfully dispensed; if dispensing fails, no debit shall occur.

ATM-6 The ATM **shall** retract cash that is not taken within 30 s and record the event.

ATM-7 The ATM **should** offer a receipt showing the amount withdrawn and remaining balance.

(c) Internet banking – internal funds transfer

TRF-1 The system **shall** allow an authenticated customer to transfer funds from one of their accounts to any account held at the same bank.

TRF-2 The customer **shall** specify the source account, destination account number, amount and an optional reference.

TRF-3 The system **shall** display the name of the destination account holder and ask the customer to confirm before executing the transfer.
Rationale: reduces transfers to wrong accounts.

TRF-4 The system **shall** reject transfers that exceed the available balance or the customer's daily transfer limit.

TRF-5 Transfers above a bank-defined threshold **shall** require a one-time password as second-factor authentication.

TRF-6 The transfer **shall** be applied to both accounts atomically and immediately, and a confirmation with a transaction ID shown.

TRF-7 The system **should** allow the customer to save frequent destination accounts as payees.

Exercise 4.6

Exercise 4.6

Suggest how an engineer responsible for drawing up a system requirements specification might keep track of the relationships between functional and non-functional requirements.

Answer. Non-functional requirements often affect many functional requirements, and one NFR can generate several functional requirements *[Ref. Sec. 4.1.2].* To keep track:

1. **Unique identification** – give every requirement a unique ID (e.g. FR-12, NFR-PERF-3) so it can be cross-referenced *[Ref. Sec. 4.6.1]*.

2. **Traceability matrix** – build a table with functional requirements as rows and non-functional requirements as columns; mark each cell where the NFR constrains the FR ("D" for depends, "R" for weaker relation). This makes impact analysis easy when either changes *[Ref. Sec. 4.6.1, 4.6.2]*.

3. **Explicit links in the specification** – in each structured specification form *[Ref. Sec. 4.4.2]*, include a field "Related NFRs", and for each NFR list the FRs it constrains or was derived from.

4. **Record rationale and source** – note why each NFR exists and which stakeholder proposed it, so conflicts can be traced to their origin.

5. **Tool support** – use a requirements management tool (e.g. IBM DOORS) or at least a shared spreadsheet/database that stores requirements and links and can report affected requirements automatically *[Ref. Sec. 4.6.1]*.

Exercise 4.7

Exercise 4.7

Using your knowledge of how an ATM is used, develop a set of use cases that could serve as a basis for understanding the requirements for an ATM system.

Answer. Use cases identify the actors in an interaction and name the type of interaction; each is then described in more detail *[Ref. Sec. 4.4.3]*. Scenarios can be written for each *[Ref. Sec. 4.3.2]*. **Actors:** Bank customer, Bank system (account database), ATM operator/maintenance staff, Card issuer network.

Use case	Actors	Description
Validate user	Customer, Bank	Customer inserts card and enters PIN; system checks card validity and PIN; card retained after 3 failed attempts. Included in all customer use cases.
Withdraw cash	Customer, Bank	Customer selects amount; bank checks balance and limits; card returned; cash dispensed; account debited; optional receipt.
Check balance	Customer, Bank	Displays or prints current and available balance.
Deposit cash/cheque	Customer, Bank	Customer inserts notes or cheque; ATM counts/scans; amount confirmed; account credited (cheques pending clearance).
Transfer funds	Customer, Bank	Transfer between the customer's linked accounts.
Change PIN	Customer, Bank	Customer enters old PIN and new PIN twice; bank updates PIN.
Print mini-statement	Customer, Bank	Prints the last 10 transactions.
Use foreign card	Customer, Card network	Card of another bank is validated through the inter-bank network; fee shown before confirmation.
Replenish ATM	Operator	Operator authenticates, reloads cash cassettes and paper, records the amounts loaded.
Run maintenance	Operator	Diagnostics, clear jams, retrieve retained cards and deposits.
Report status	ATM, Bank	ATM periodically reports cash level and faults to the bank.

Example detailed use case – Withdraw cash

Actors Customer, Bank system.

Pre-condition
Customer validated (*Validate user*).

Normal flow 1. Customer chooses "Withdraw". 2. Selects account and amount. 3. ATM requests authorization from bank. 4. Bank approves. 5. Card returned. 6. Cash dispensed. 7. Account debited; receipt printed if requested.

Alternatives Insufficient funds or limit exceeded → message, return to menu. ATM lacks notes → suggest a different amount. Cash not taken in 30 s → retracted, transaction reversed.

Post-condition
Account balance reduced by the dispensed amount; transaction logged.

Exercise 4.8

Exercise 4.8

To minimize mistakes during a requirements review, an organization decides to allocate two scribes to document the review session. Explain how this can be done.

Answer. In a requirements review a team of customer and developer staff reads the document and checks for errors, anomalies and inconsistencies, which must be recorded before being negotiated *[Ref. Sec. 4.5]*. Two scribes can be used as follows:

1. **Divide responsibilities.** Scribe A records the *problems* raised (requirement ID, description, type – ambiguity, inconsistency, omission, unrealistic, unverifiable – and who raised it); Scribe B records the *discussion and decisions* (agreed resolution, actions, owner, deadline). Alternatively, they may split by document section.
2. **Use a common template** – both use the same requirement IDs and problem categories (validity, consistency, completeness, realism, verifiability *[Ref. Sec. 4.5]*) so records can be merged.
3. **Independent overlapping notes** – for important items both scribes record independently; differences highlight possible misunderstandings.
4. **Reconcile after the session** – the scribes compare and merge their notes into one review report soon after the meeting, resolving discrepancies (with the moderator if needed).
5. **Confirm with participants** – the consolidated report is circulated to reviewers, customer and developers for confirmation; it then feeds into the change management process *[Ref. Sec. 4.6.2]*.

Exercise 4.9

Exercise 4.9

When emergency changes have to be made to systems, the system software may have to be modified before changes to the requirements have been approved. Suggest a model of a process for making these modifications that will ensure that the requirements document and the system implementation do not become inconsistent.

Answer. The normal change process is *problem analysis and change specification → change analysis and costing → change implementation* *[Ref. Sec. 4.6.2]*. If urgent changes are made to the system first, it is easy to forget to update the requirements document, so it must be updated as soon as possible *[Ref. Sec. 4.6.2]*. A suggested emergency process:

1. **Raise an emergency change request (ECR)** – record the problem, the affected requirement IDs and the reason the change is urgent. The ECR is logged in the change management tool with status *"Emergency – requirements pending"*.

2. **Fast authorization** – a small authority (e.g. project manager plus customer representative) approves the emergency fix, instead of the full change board.

3. **Implement the fix** – code is modified; every commit references the ECR ID, and a brief draft of the changed requirement is attached to the ECR.

4. **Mandatory follow-up** – the ECR cannot be closed until the requirements document is updated. The system is labelled as a "temporary" release.

5. **Retrospective change analysis** – within a fixed deadline (e.g. 5 working days) the change goes through normal analysis and costing using traceability links *[Ref. Sec. 4.6.1]* to find other affected requirements and design.

6. **Approve and update** – the change board formally approves (or requests rework of) the change; the requirements document, design and tests are updated, and the ECR is closed.

7. **Audit** – periodic reports list all open ECRs, so no emergency change remains undocumented.

Exercise 4.10

Exercise 4.10

You have taken a job with a software user who has contracted your previous employer to develop a system for them. You discover that your company's interpretation of the requirements is different from the interpretation taken by your previous employer. Discuss what you should do in such a situation. You know that the costs to your current employer will increase if the ambiguities are not resolved. However, you also have a responsibility of confidentiality to your previous employer.

Answer. This is a professional-ethics problem. Requirements are often ambiguous and different stakeholders interpret them differently *[Ref. Sec. 4.4.1]*; resolving that is the purpose of requirements validation and negotiation *[Ref. Sec. 4.3, 4.5]*. A reasonable course of action:

1. **Do not disclose confidential information** of the previous employer (internal designs, documents, costings, strategies). Confidentiality obligations continue after leaving.

2. **Raise the issue using only public/shared information.** The requirements document is shared between the customer (your current employer) and the contractor, so you can legitimately point out that particular requirements are *ambiguous* and could be interpreted in more than one way, without revealing how the previous employer interprets them.

3. **Recommend a formal requirements review** *[Ref. Sec. 4.5]* with the contractor, in which both parties state their interpretation of the ambiguous requirements; the differences will then appear naturally and can be negotiated *[Ref. Sec. 4.3.1]*.

4. **Declare the conflict of interest** to your current manager and, if necessary, ask not to be involved in negotiations where your previous knowledge would be relevant.

5. **If unsure, seek advice** (e.g. professional code of ethics, legal department) before acting.

In this way you protect your current employer's interests (the ambiguity gets resolved early, when it is cheapest) while honouring your duty of confidentiality to the previous employer.

Reference

I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016, Chapter 4 "Requirements engineering" (Sec. 4.1 Functional and non-functional requirements; 4.2 RE processes; 4.3 Requirements elicitation; 4.4 Requirements specification; 4.5 Requirements validation; 4.6 Requirements change).